

Day 1 Technical Report

Reproducibility, Dependency Management (Poetry), and Data Ops (DVC)

Hamady Gackou
Master Data Science & AI

January 12, 2026

Abstract

This document details the foundational engineering choices made on Day 1 of the *ClusterLLM-Local* project. Beyond setting up the environment on a GCP L4 GPU instance, this report analyzes the strategic decisions to prioritize **Reproducibility** over speed. It provides a technical justification for choosing **Poetry** over standard `pip/venv` for dependency management and **DVC** for data versioning. The day concluded with the successful deployment of a "Tracer Bullet" pipeline using the *Bank77* dataset.

Contents

1	The Core Philosophy: Reproducibility	1
1.1	What is Reproducibility?	2
1.2	Why is it critical for this project?	2
2	Dependency Management: Why Poetry?	2
2.1	The Limitation of Standard Virtual Environments	2
2.2	The Poetry Solution	2
3	Data Operations: Data Version Control (DVC)	3
3.1	The Problem with Git and Data	3
3.2	How DVC Works	3
4	Implementation Log (Day 1)	3
4.1	1. Environment Setup	3
4.2	2. Connecting Storage	3
4.3	3. The Tracer Bullet (Bank77)	4
5	Conclusion & Roadmap	4

1 The Core Philosophy: Reproducibility

Before writing code, we defined the primary constraint of this research: **Reproducibility**.

1.1 What is Reproducibility?

In the context of Machine Learning, reproducibility is the ability for an independent researcher (or a Jury member) to recreate the exact same results (metrics, graphs, models) given the same input data and code.

1.2 Why is it critical for this project?

- **Scientific Rigor (Master's Defense):** If the code works "only on my machine" because of a hidden manual configuration, the scientific contribution is null.
- **The "Dependency Hell" Risk:** In 6 months, libraries like transformers or torch will update. Without strict locking, the code might break or produce different results.
- **PhD Track Requirement:** Top-tier conferences (ICLR, NeurIPS) require a "Reproducibility Checklist". Setting this up on Day 1 ensures compliance.

2 Dependency Management: Why Poetry?

To ensure reproducibility, we rejected the standard `venv + pip` workflow in favor of **Poetry**.

2.1 The Limitation of Standard Virtual Environments

A standard Python workflow uses 'virtualenv' and a 'requirements.txt' file generated by 'pip freeze'.

- **The Problem:** 'pip' installs packages sequentially. If Package A needs 'numpy<2.0' and Package B needs 'numpy>=2.0', pip might install conflicting versions without warning, leading to runtime crashes.
- **No Locking:** A 'requirements.txt' often lists top-level packages (e.g., 'pandas'). It rarely locks the hundreds of sub-dependencies. A user installing it tomorrow might get a different sub-dependency version than the one I used today.

2.2 The Poetry Solution

Poetry is a dependency manager and packaging tool that solves these issues.

1. **Deterministic Resolution:** Before installing anything, Poetry solves the mathematical graph of dependencies to ensure no conflicts exist.
2. **The Lock File** (`poetry.lock`): This is the "Source of Truth". It records the exact SHA-256 hash of every single file installed.
3. **Separation of Concerns:** It separates *Production* dependencies (PyTorch) from *Development* dependencies (PyTest, Black) in the 'pyproject.toml' file.

Analogy

Standard Venv: Throwing ingredients into a pot and hoping the soup tastes good.

Poetry: A strict recipe with exact measurements and brand names for every ingredient, checked by a chemist before cooking.

3 Data Operations: Data Version Control (DVC)

Just as Poetry locks the code environment, DVC locks the data.

3.1 The Problem with Git and Data

Git computes differences between lines of text. It cannot handle binary large objects (BLOBs) like CSVs or Model Weights. Storing a 500MB dataset in Git slows down the repository and hits storage limits.

3.2 How DVC Works

DVC acts as a pointer system (like a hyperlink to a file).

1. **The Workspace:** We put the actual data in 'data/raw/bank77.csv'.
2. **The Hashing:** `dvc add` calculates the MD5 hash of the file.
3. **The Pointer:** DVC replaces the data with a small text file: 'bank77.csv.dvc'.
4. **The Remote:** The actual heavy file is pushed to Google Drive (our "Object Storage").

Feature	Git	DVC
Handles	Code (Text)	Data & Models (Binary)
Storage	GitHub Servers	Google Drive / S3 / GCP
Tracking Method	Line-by-line Diff	MD5 Hash
Repo Size	Small (<100MB)	Small (Contains only pointers)

Table 1: Comparison of Git vs. DVC roles in the project.

4 Implementation Log (Day 1)

4.1 1. Environment Setup

We initialized the deterministic environment.

```

1 # Core dependencies for the research paper (CUDA 11.8 compatible)
2 poetry add torch torchvision torchaudio --index-url https://download.
  pytorch.org/whl/cu118
3 # MLOps Stack
4 poetry add dvc dvc-gdrive mlflow hydra-core

```

Listing 1: Poetry Setup

4.2 2. Connecting Storage

We linked the local GPU instance to Google Drive.

```

1 dvc init
2 dvc remote add -d myremote gdrive://<GDRIVE_ID>
3 git commit -m "chore: setup dvc remote"

```

Listing 2: DVC Remote Setup

4.3 3. The Tracer Bullet (Bank77)

To validate the infrastructure without incurring the cost of processing all 14 datasets, we implemented a single dataset pipeline.

```
1 # 1. Download Data
2 poetry run python src/data/loader.py
3
4 # 2. Version Control Data (DVC)
5 dvc add data/raw/bank77_train.csv
6 dvc push # Sends to Google Drive
7
8 # 3. Version Control Code (Git)
9 git add data/raw/*.dvc
10 git commit -m "data: add bank77 dataset"
11 git push
```

Listing 3: Workflow Execution

5 Conclusion & Roadmap

We have successfully established a **Reproducible Infrastructure**.

- Code is locked via `poetry.lock`.
- Data is locked via `.dvc` files.
- History is tracked via `git`.

Next Step (Day 2): Containerization. We will wrap this entire ecosystem into a **Docker** container to guarantee that the operating system libraries (CUDA drivers) are also reproducible.